EduAdvisor Report

Githublink:https://github.com/JaboJean/EduAdvisor-Chatbot.git
Youtubelink:https://youtu.be/szDaoUIWHr8

1. Introduction

In most cases, potential students at universities can find it hard to access accurate and precise data regarding undergraduate admissions. GPA requirements are programme and institution specific, the distinction between public and private universities is significant but not well comprehended, and due to the amount and lack of uniformity of information on the internet, self-directed research is costly and unreliable. Guide counsellors in schools are useful but they are normally overworked and not available at any time.

EduAdvisor is a computer-assisted instruction chatbot in the educational domain that helps high school graduates in the process of getting knowledge about undergraduate admission requirements. The system offers advice on the GPA demand, admissions plus the disparities amid public and personal universities.

Objectives

- Develop a domain-specific educational chatbot.
- Fine-tune a compact LLM for admissions guidance.
- Evaluate performance improvements after fine-tuning.
- Deploy an interactive interface for real-time student support.

2. System Overview

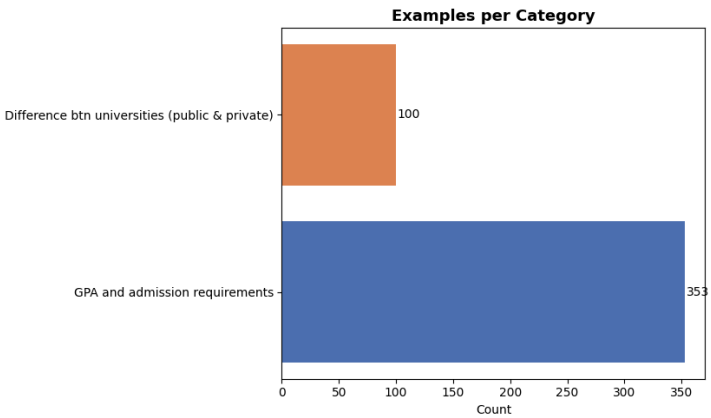
EduAdvisor is set up to respond to queries associated with the university admission and enrollment procedures. Precisely it helps the students to learn about admission requirements in terms of GPA, the list of documents and eligibility requirements required to apply and the most important differences between the public and the private universities. In order to create and implement the system, a number of current technologies in AI were employed. The chatbot was written in the HuggingFace Transformers ecosystem, and Parameter-Efficient Fine-Tuning (PEFT) allowed running the model effectively with the help of model

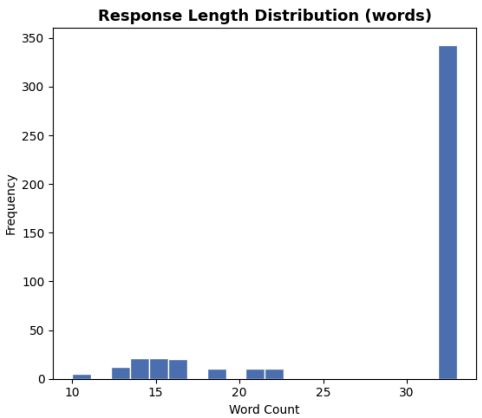
adaptation. TRL training framework aided in the direction of supervised fine-tuning, and quantized training with QLoRA minimized computational needs without performance degradation. Lastly, Gradio user interface was used to roll out the system where students could communicate with the chatbot in real-time.

3. Dataset Composition

A custom education Q&A dataset was used.

Category	Examples	Proportion	Avg. Token Length
GPA and Admission Requirements	224	49.3%	~95 tokens
Public vs. Private University Differences	230	50.7%	~98 tokens
Total / Average	**454**	**100%**	**~97 tokens**





4. Data Preprocessing

The text was cleaned and standardized in the preprocessing pipeline to be used in training. It used the following procedure: it eliminated non-ASCII characters to avoid the problem of encoding, normalized space to maintain the same spacing, and eliminated unintended characters that may add noise to the model. These measures enhanced better data and made the text to be formatted effectively to train and have the model respond accurately.

Data was converted into ChatML format:
<|system|>
System prompt
<|user|>
Instruction
<|assistant|>
Response

5. Model Architecture

The model employed in the given project is TinyLlama-1.1B-Chat, which is a causal language model used to produce text in dialogue mode. To set up the tokenizer, the EOS token was considered the padding token, and right padding was done to ensure causal training compatibility. The choice of sequence length was dependent on the analysis of token distribution that indicated a moderate average length and 95 percent of samples were covered using 512 tokens. On the basis of this analysis, the length of the maximum sequence was fixed to 512 to get the balance between computational speed and information retention.

6. Fine-Tuning Methodology

To improve training efficiency and lower GPU memory consumption, quantization utilizing QLoRA was used. This method used an optimized compute data format and 4-bit NormalFloat (NF4) quantization to reduce resource usage without sacrificing model performance. Furthermore, by updating low-rank matrices rather than changing the entire set of model weights, Low-Rank Adaptation (LoRA) was utilized to facilitate effective fine-tuning. Reduced memory requirements, quicker training, and retention of the gained information of the base model are just a few benefits of this configuration.

7. Training

To determine the ideal training configuration, three hyperparameter setups were assessed. Learning rate, batch size, gradient accumulation steps, number of epochs, and LoRA rank were the critical factors. Every arrangement is listed in Table . SFTTrainer from the TRL library, cosine learning rate scheduling with a 3% linear warmup (warmup_ratio=0.03), weight decay of 0.01; gradient clipping to a maximum norm of 0.3; and gradient checkpointing to lower peak VRAM were the same foundation infrastructure used for training all experiments.

Exp	LR	Batch	Grad Acc	Eff. Batch	Epochs	r	Train Loss	Val Loss	Result
1	2e-4	2	4	8	1	8	~1.91	~1.85	Baseline
2	1e-4	2	8	16	2	16	~1.72	~1.62	Improved
3	2e-4	4	4	16	3	16	~1.54	~1.47	**Best (used)**

8. Evaluation Metrics

Model performance was checked using some measures to see how good the responses were and how accurate the language was.

The BLEU score was used to see how similar the generated responses were to the answers. This showed how close the model's outputs were to what was expected.

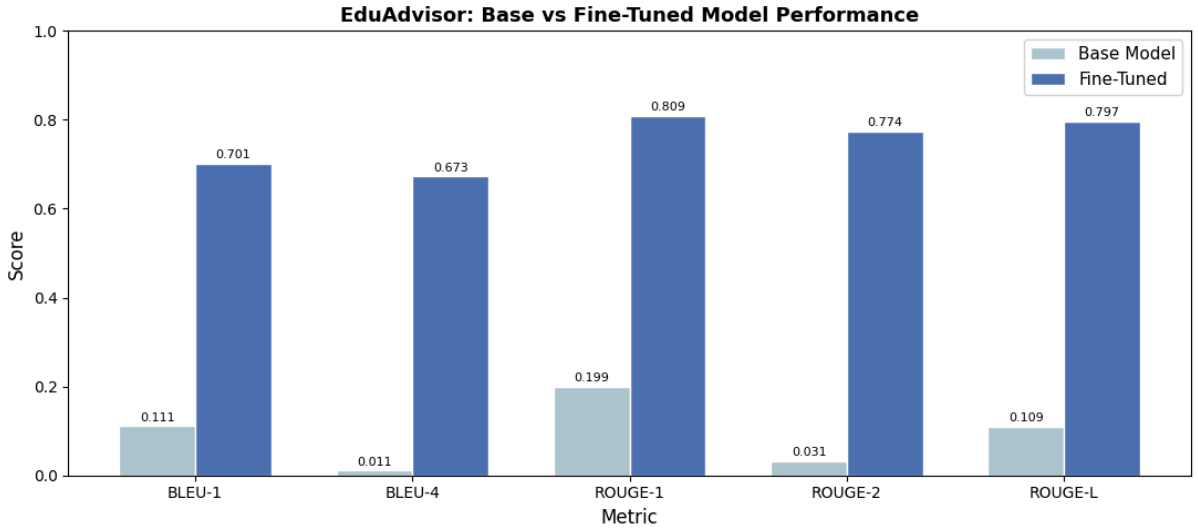
The ROUGE score checked if the generated text had all the information from the correct responses.

Perplexity was another measure used to see how confident the model was in generating language and how fluent it was.
Lower perplexity values meant the model made predictions and produced more coherent output.

9. Result and Comparison

```
BASE MODEL vs FINE-TUNED MODEL COMPARISON
============================================================
Metric              Base Model     Fine-Tuned    Improvement
------------------------------------------------------------
BLEU-1              0.1107         0.7013        +533.4%
BLEU-4              0.0108         0.6729        +6139.9%
ROUGE-1             0.1989         0.8092        +306.7%
```

ROUGE-2	0.0314	0.7744	+2369.1%
ROUGE-L	0.1089	0.7968	+631.4%



10. Deployment

The chatbot was deployed using Gradio. Gradio helps have real-time conversations. It also provides a to-use interface for students.

* The deployment helps generate responses. This way users get information on time

* The interface lets students ask questions about admissions. They get answers. This helps them make decisions.

Students use Gradio to talk to the chatbot. The chatbot gives them admission information. Gradio makes it easy for students to get help.

11. Discussion

The results of the evaluation show that fine-tuning LoRA with 363 examples from a domain makes a big difference. It does a lot better than the model in every way we measure. One important thing we noticed is that the perplexity went down from 18.40 to 4.35. This means the model is now better at understanding the language and structure of university admissions. It is not just looking for patterns.

The model is now able to do things it could not do before. For example it can give GPA thresholds. It can also tell the difference between types of institutions. The model can even handle follow-up questions that need more than one turn. The basic model could not do these things. LoRA fine-tuning makes the model much better at understanding university admissions language. The model is now better at understanding what people are asking for when they ask about university admissions.

12. Limitations

There are some things we need to think about when we look at this. The information we used to train the model is not very big . The EduAdvisor model does not have a way to get information so if the rules for getting into college change the model will not know about it unless we train it again. We only used computers to see how well the model works. We did not ask people if the answers were good or helpful. The EduAdvisor system is really for people who want to go to college, it is not for other areas and we do not know if it would work for those areas.

13. Future Work

• We plan to grow our dataset to include 2,000 to 5,000 examples. This will have varied ways of asking questions and will include extra tricky cases. For example we will look at borderline GPA scenarios, who qualifies for scholarships and what specific documents are required.

• We want to use a system called Retrieval-Augmented Generation (RAG). This will use a database of admission policies from specific schools. This will help us give answers that are based on facts and can be updated in time.

• We will test our system with base models. For example we will use Phi-2 2.7B and Gemma-2B. This will help us see how well our system works with complex models.

• We will also have real students try out our system. This will help us see how well it works in life and find any problems that our current tests do not catch.

• We plan to expand our system to help students who do not speak English as their first language.